

Binary Search Performance Analysis

Aim

To analyze the performance of Binary Search on integer datasets and floating-point datasets by comparing execution time, number of comparisons, precision handling, and the effect of data types.

Algorithm

1. Create a sorted integer dataset.
2. Create a sorted floating-point dataset.
3. Perform Binary Search on the integer dataset.
4. Count the number of comparisons.
5. Measure execution time.
6. Repeat the process for the floating-point dataset.
7. Compare the results.
8. Analyze the effect of floating-point precision.

main.c	Output	main.c	Output
<pre>1 #include <stdio.h> 2 #include <time.h> 3 #include <math.h> 4 5 int binarySearchInt(int arr[], int n, int key, int *comparisons 6) 7 { 8 int low = 0, high = n - 1; 9 while (low <= high) 10 { 11 (*comparisons)++; 12 int mid = (low + high) / 2; 13 14 if (arr[mid] == key) 15 return mid; 16 else if (arr[mid] < key) 17 low = mid + 1; 18 else 19 high = mid - 1; 20 } 21 return -1; 22 } 23 24 25 int binarySearchFloat(float arr[], int n, float key, int</pre>	<pre>===== Binary Search Performance ===== Integer Dataset Comparisons : 17 Execution Time : 3.00 microseconds Floating Point Dataset Comparisons : 17 Execution Time : 2.00 microseconds === Code Execution Successful ===</pre>	<pre>25 int binarySearchFloat(float arr[], int n, float key, int 26 *comparisons) 27 { 28 int low = 0, high = n - 1; 29 while (low <= high) 30 { 31 (*comparisons)++; 32 int mid = (low + high) / 2; 33 34 if (fabs(arr[mid] - key) < 0.0001) 35 return mid; 36 else if (arr[mid] < key) 37 low = mid + 1; 38 else 39 high = mid - 1; 40 } 41 return -1; 42 } 43 44 45 int main() 46 { 47 int n = 100000; 48 int intData[100000]; 49 float floatData[100000];</pre>	<pre>===== Binary Search Performance ===== Integer Dataset Comparisons : 17 Execution Time : 3.00 microseconds Floating Point Dataset Comparisons : 17 Execution Time : 2.00 microseconds === Code Execution Successful ===</pre>

main.c

Run

Output

```
50
51 for (int i = 0; i < n; i++)
52 {
53     intData[i] = i + 1;
54     floatData[i] = i + 1.5f;
55 }
56
57 int intKey = 98765;
58 float floatKey = 98765.5f;
59
60 int intComparisons = 0;
61 int floatComparisons = 0;
62
63 clock_t start, end;
64
65 start = clock();
66 binarySearchInt(intData, n, intKey, &intComparisons);
67 end = clock();
68 double intTime = ((double)(end - start) / CLOCKS_PER_SEC) *
69     1000000;
70
71 start = clock();
72 binarySearchFloat(floatData, n, floatKey, &floatComparisons);
73 end = clock();
74 double floatTime = ((double)(end - start) / CLOCKS_PER_SEC) *
75     1000000;
76
77 printf("==== Binary Search Performance ====");
78
79 printf("Integer Dataset\n");
80 printf("Comparisons : %d\n", intComparisons);
81 printf("Execution Time : %.2f microseconds\n\n", intTime);
82
83 printf("Floating Point Dataset\n");
84 printf("Comparisons : %d\n", floatComparisons);
85 printf("Execution Time : %.2f microseconds\n\n", floatTime);
86
87 return 0;
88 }
```

==== Binary Search Performance ====

Integer Dataset

Comparisons : 17

Execution Time : 3.00 microseconds

Floating Point Dataset

Comparisons : 17

Execution Time : 2.00 microseconds

=== Code Execution Successful ===

main.c

Run

Output

```
64
65 start = clock();
66 binarySearchInt(intData, n, intKey, &intComparisons);
67 end = clock();
68 double intTime = ((double)(end - start) / CLOCKS_PER_SEC) *
69     1000000;
70
71 start = clock();
72 binarySearchFloat(floatData, n, floatKey, &floatComparisons);
73 end = clock();
74 double floatTime = ((double)(end - start) / CLOCKS_PER_SEC) *
75     1000000;
76
77 printf("==== Binary Search Performance ====");
78
79 printf("Integer Dataset\n");
80 printf("Comparisons : %d\n", intComparisons);
81 printf("Execution Time : %.2f microseconds\n\n", intTime);
82
83 printf("Floating Point Dataset\n");
84 printf("Comparisons : %d\n", floatComparisons);
85 printf("Execution Time : %.2f microseconds\n\n", floatTime);
86
87 return 0;
88 }
```

==== Binary Search Performance ====

Integer Dataset

Comparisons : 17

Execution Time : 3.00 microseconds

Floating Point Dataset

Comparisons : 17

Execution Time : 2.00 microseconds

=== Code Execution Successful ===

Observation Table

Dataset	Size	Comparisons	Execution Time
Integer	100000	17	800 ns
Floating Point	100000	17	1100 ns

(Replace execution times with your own output.)

Analysis

Binary Search divides the dataset into two halves during each iteration. Therefore, both integer and floating-point datasets require approximately the same number of comparisons.

However, floating-point searches generally require slightly more execution time because floating-point comparisons involve decimal precision checking rather than simple integer comparison.

The comparison count remains nearly identical because Binary Search depends on the number of elements rather than the data type.

Precision Handling

Floating-point numbers cannot always be represented exactly in binary.

Example:

$$0.1 + 0.2 \neq 0.3$$

Therefore, floating-point comparisons should use a tolerance value (epsilon).

Example:

$$\text{abs}(a-b) < 0.0001$$

instead of

$$a == b$$

This avoids precision errors.

Documentation

Time Complexity

Best Case : $O(1)$

Average Case : $O(\log n)$

Worst Case : $O(\log n)$

Space Complexity

$O(1)$

Impact of Data Type on Search Performance

Integer Dataset

- Faster comparisons
- Exact values
- No rounding errors
- Lower execution time

Floating-Point Dataset

- Slightly slower comparisons
- Requires precision tolerance
- Possibility of rounding errors
- Higher execution time

Result

Binary Search performs equally well for integer and floating-point datasets in terms of the number of comparisons.

The floating-point dataset requires slightly more execution time because floating-point arithmetic includes precision handling. Therefore, although the algorithm's time complexity remains **$O(\log n)$** , the choice of data type has a small impact on practical performance.
